

Network System Security

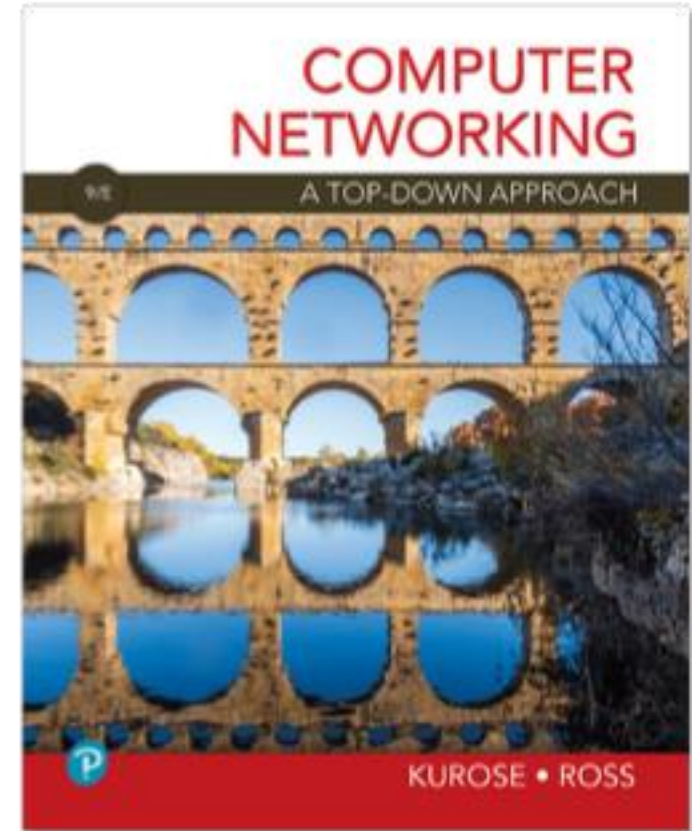
Priyansh Singh

PhD Candidate, Computer Science and Engineering

priyanshs.me

Books and References

- Chapter 8: Security from Computer Networking: A Top-Down Approach, 9th edition. Jim Kurose, Keith Ross
- Cryptography and Network Security: Principles and Practice 8th Edition, William Stallings



Computer Networking: A Top-Down Approach

9th edition

Jim Kurose, Keith Ross
Pearson, 2025

Outline

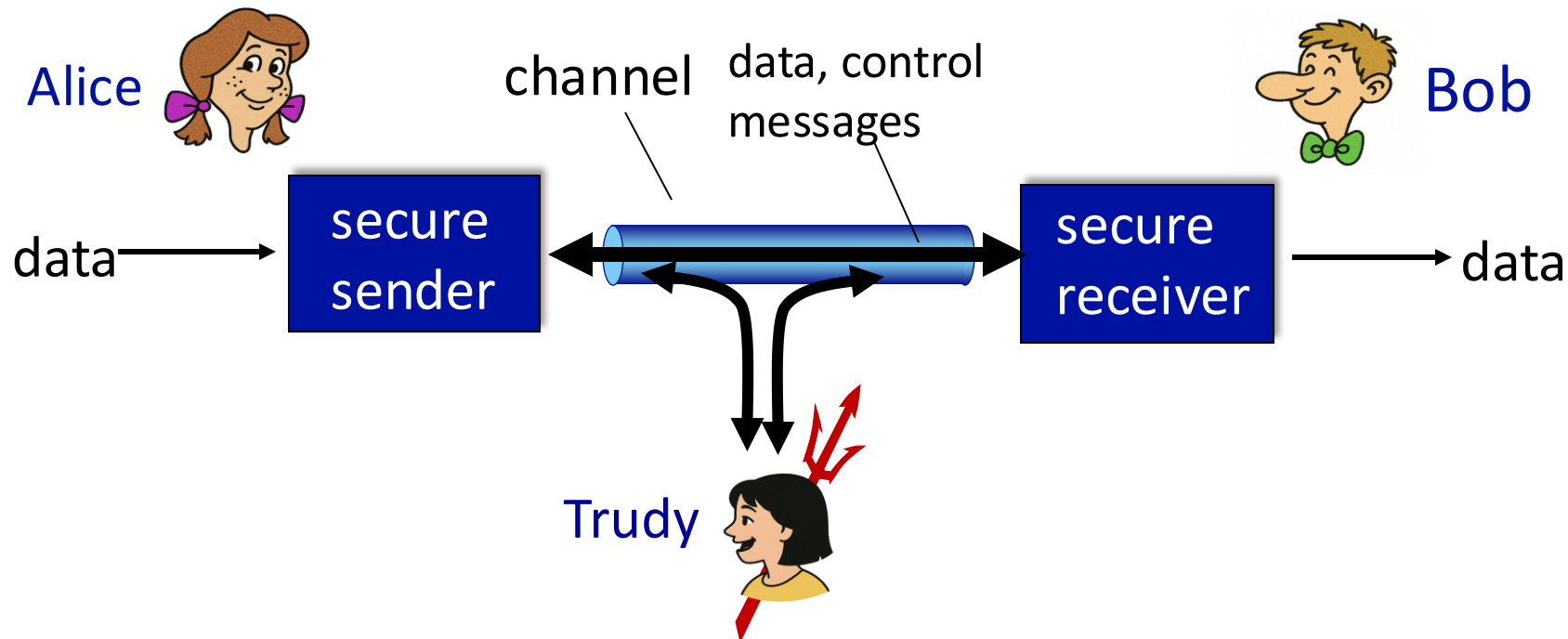
- Our Actors
- Public Key Cryptography
 - Diffie Hellman Key Exchange
 - RSA Algorithm
- Authentication
- Deployed in wireless and mobile networks

Outline

- Our Actors
- Public Key Cryptography
 - Diffie Hellman Key Exchange
 - RSA Algorithm
- Authentication
- Deployed in wireless and mobile networks

Friends and enemies: Alice, Bob, Trudy

- well-known in network security world
- Bob, Alice (lovers!) want to communicate “securely”
- Trudy (intruder) may intercept, delete, add messages



Friends and enemies: Alice, Bob, Trudy

Who might Bob and Alice be?

- ... well, *real-life* Bobs and Alices!
- Web browser/server for electronic transactions (e.g., on-line purchases)
- on-line banking client/server
- DNS servers
- BGP routers exchanging routing table updates
- other examples?

There are bad guys (and girls) out there!

Q: What can a “bad guy” do?

A: A lot!

- **eavesdrop**: intercept messages
- actively **insert** messages into connection
- **impersonation**: can fake (spoof) source address in packet (or any field in packet)
- **hijacking**: “take over” ongoing connection by removing sender or receiver, inserting himself in place
- **denial of service**: prevent service from being used by others (e.g., by overloading resources)

Outline

- Our Actors
- Public Key Cryptography
 - Diffie Hellman Key Exchange
 - RSA Algorithm
- Authentication
- Deployed in wireless and mobile networks

Breaking an encryption scheme

- **cipher-text only attack:**
Trudy has ciphertext she can analyze

- **two approaches:**
 - brute force: search through all keys
 - statistical analysis

- **known-plaintext attack:**
Trudy has plaintext corresponding to ciphertext
 - *e.g.*, in monoalphabetic cipher, Trudy determines pairings for a,l,i,c,e,b,o,
- **chosen-plaintext attack:**
Trudy can get ciphertext for chosen plaintext

Requirements

- How should a public key scheme work?
- Three main conditions
 - It must be computationally easy to encrypt or decrypt a message given the appropriate key
 - It must be computationally infeasible to derive the private key from the public key
 - It must be computationally infeasible to determine the private key from chosen plaintext attack
 - Attacker can pick any message, have it encrypted, and obtain the ciphertext

Hard problems

- Public key cryptosystems are based on hard problems
- Discrete Logarithm Problem (DLP)
- Given:
 - Multiplicative group G
 - Element a in G
 - Output b
- Find:
 - Unique solution to $ax = b$ in G
 - x is $\log_a b$
- No polynomial time algorithm exists to solve this*

*On classical computers

Public Key Cryptography

symmetric key crypto:

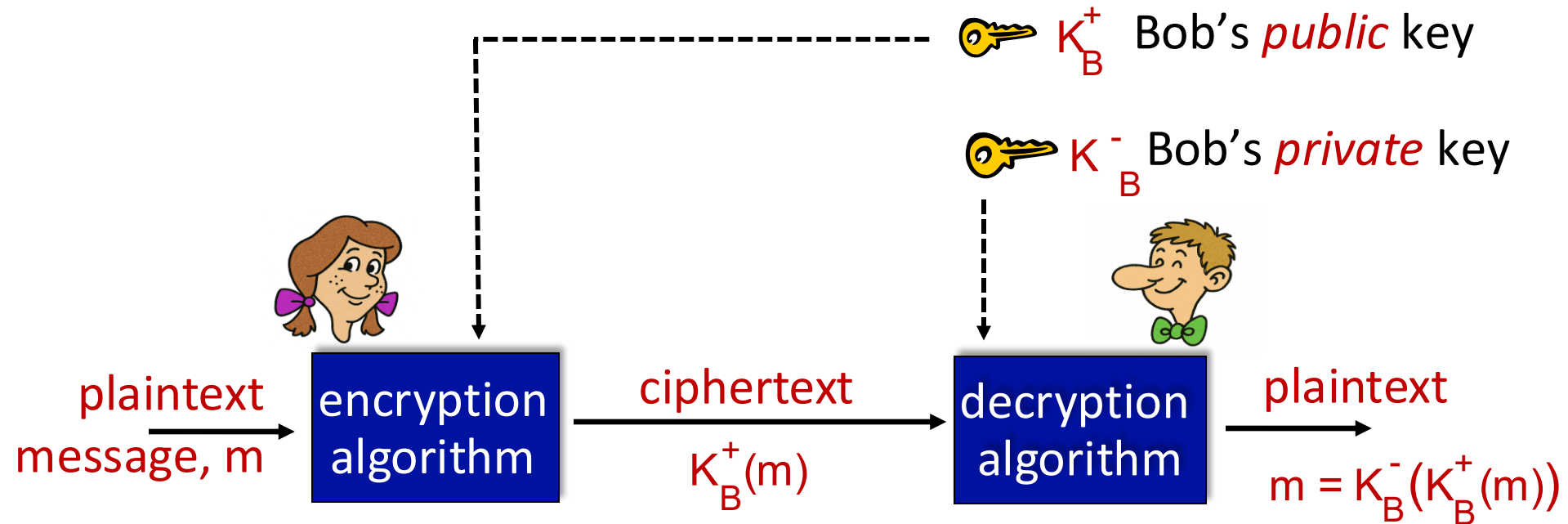
- requires sender, receiver know shared secret key
- Q: how to agree on key in first place (particularly if never “met”)?

public key crypto

- *radically* different approach [Diffie-Hellman76, RSA78]
- sender, receiver do *not* share secret key
- *public* encryption key known to *all*
- *private* decryption key known only to receiver



Public Key Cryptography



Wow - public key cryptography revolutionized 2000-year-old (previously only symmetric key) cryptography!

- similar ideas emerged at roughly same time, independently in US and UK (classified)

Public key encryption algorithms

requirements:

① need $K_B^+(\cdot)$ and $K_B^-(\cdot)$ such that

$$K_B^-(K_B^+(m)) = m$$

② given public key K_B^+ , it should be impossible to compute private key K_B^-

RSA: Rivest, Shamir, Adelson algorithm

Prerequisite: modular arithmetic

- $x \bmod n$ = remainder of x when divide by n

- facts:

$$[(a \bmod n) + (b \bmod n)] \bmod n = (a+b) \bmod n$$

$$[(a \bmod n) - (b \bmod n)] \bmod n = (a-b) \bmod n$$

$$[(a \bmod n) * (b \bmod n)] \bmod n = (a*b) \bmod n$$

- thus

$$(a \bmod n)^d \bmod n = a^d \bmod n$$

- example: $x=14$, $n=10$, $d=2$:

$$(x \bmod n)^d \bmod n = 4^2 \bmod 10 = 6$$

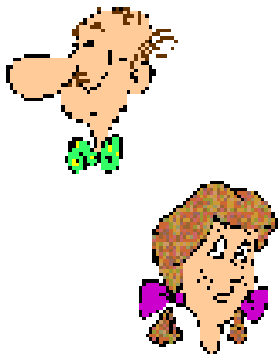
$$x^d = 14^2 = 196 \quad x^d \bmod 10 = 6$$

Using Public-Key Cryptography to Agree on a Shared Key

Problem: for Bob and Alice to use symmetric key techniques they must agree on the shared key.

- What if they've never met before?
- *public-key cryptography can solve this problem too!*

Diffie-Hellman (DH) public/private keys: different algorithm than RSA



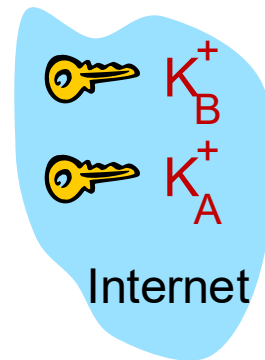
- Bob chooses **public** values: large prime number p , another large number $g (< p)$.
- Alice, Bob independently choose **private** values: S_A , S_B
- **Alice's public key:** $K_A^+ = g^{S_A} \bmod p$
- **Bob's public key:** $K_B^+ = g^{S_B} \bmod p$

Using Public-Key Cryptography to Agree on a Shared Key

Diffie-Hellman (DH) shared key derivation:

- Alice, Bob compute a common shared secret without ever explicitly exchanging this shared secret!
- Alice, Bob have DH public keys: K_A^+ , K_B^+ , secret values SA , SB

 SA
Alice's computed shared secret
(uses Bob's public key and SA):
 $(K_B^+)^{SA} \bmod p$



 SB
Bob's computed shared secret
(uses Alice's public key and SB):
 $(K_A^+)^{SB} \bmod p$

Alice's and Bob's locally computed values are equal to each other!

Diffie Hellman Key Exchange

- Developed by Martin Hellman, Ralph Merkle, and Whitfield Diffie at Stanford University in 1976
- All users share common modulus, p , and element g
 - $g \neq 0$, $g \neq 1$, and $g \neq p-1$
- Alice chooses her private key, k_A
 - Computes $K_A = g^{k_A} \bmod p$ and sends it to Bob in the clear
- Bob chooses his private key, k_B
 - Computes $K_B = g^{k_B} \bmod p$ and sends it to Alice in the clear
- When Alice and Bob want to agree on a shared key, they compute a shared secret S
 - $S_{A,B} = K_B^{k_A} \bmod p$
 - $S_{B,A} = K_A^{k_B} \bmod p$

Why does DH work?

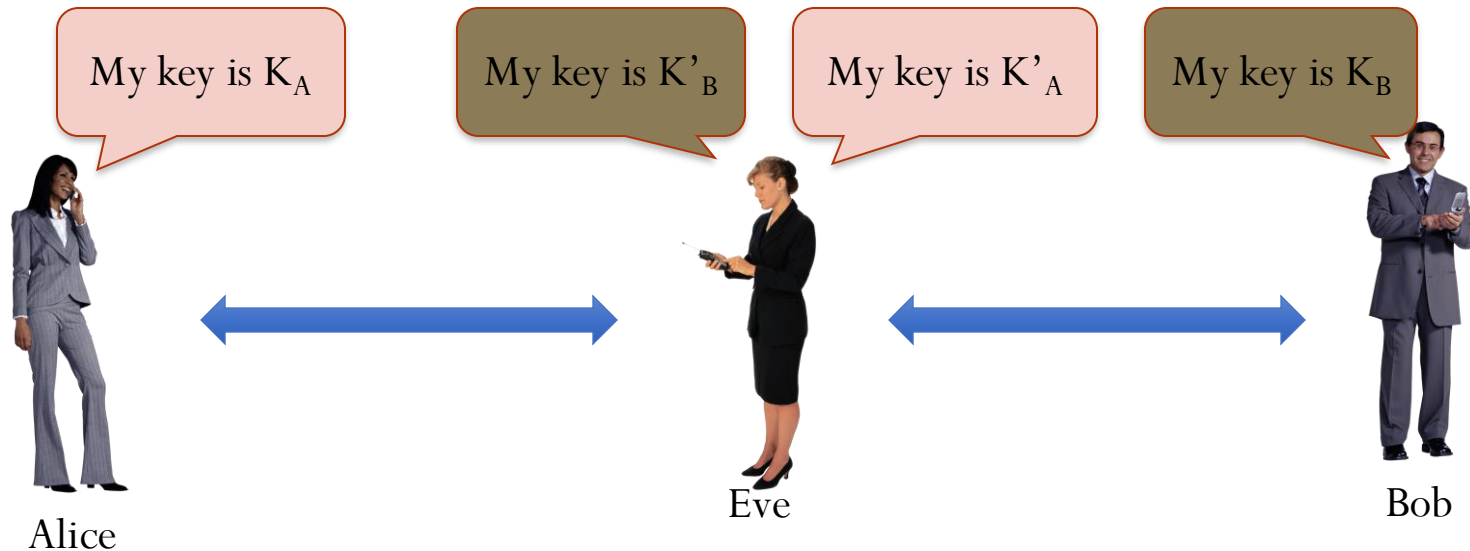
- $S_{A,B} = S_{B,A}$
- $(g^{k_A})^{k_B} \bmod p = (g^{k_B})^{k_A} \bmod p$
- Eve knows
 - g and p
 - K_A and K_B
 - Why can't Eve compute the secret?

$$S_{A,B} = K_B^{k_A} \bmod p$$
$$S_{B,A} = K_A^{k_B} \bmod p$$

- This was the first public key cryptography scheme

Could it fail?

- Eve could fool Alice and Bob
 - Man in the middle



Alice has no guarantee that the person she's establishing a key with is actually Bob

Outline

- Our Actors
- **Public Key Cryptography**
 - Diffie Hellman Key Exchange
 - **RSA Algorithm**
- Authentication
- Deployed in wireless and mobile networks

RSA: getting ready

- message: just a bit pattern
- bit pattern can be uniquely represented by an integer number
- thus, encrypting a message is equivalent to encrypting a number

example:

- $m = 10010001$. This message is uniquely represented by the decimal number 145.
- to encrypt m , we encrypt the corresponding number, which gives a new number (the ciphertext).

RSA: Creating public/private key pair

1. choose two large prime numbers p, q . (e.g., 1024 bits each)
2. compute $n = pq$, $z = (p-1)(q-1)$
3. choose e (with $e < n$) that has no common factors with z (e, z are “relatively prime”).
4. choose d such that $ed-1$ is exactly divisible by z . (in other words: $ed \bmod z = 1$).
5. *public* key is (n, e) . *private* key is (n, d) .
 $\underbrace{(n, e)}_{K_B^+}$ $\underbrace{(n, d)}_{K_B^-}$

RSA: encryption, decryption

0. given (n, e) and (n, d) as computed above
1. to encrypt message $m (< n)$, compute
$$c = m^e \bmod n$$
2. to decrypt received bit pattern, c , compute
$$m = c^d \bmod n$$

magic happens!
$$m = (\underbrace{m^e \bmod n}_c)^d \bmod n$$

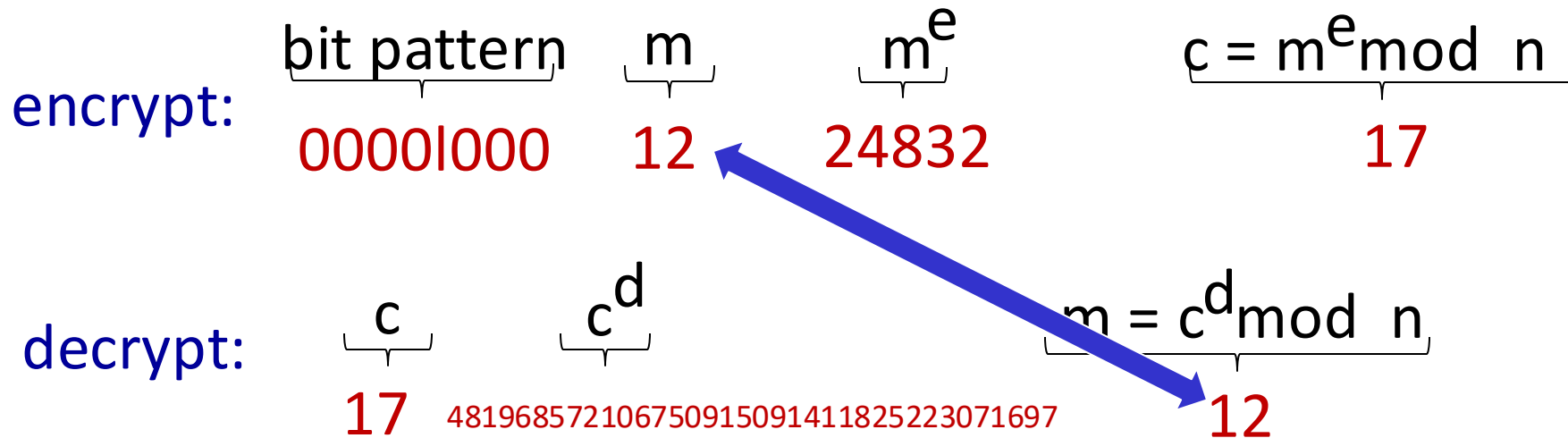
RSA example:

Bob chooses $p=5$, $q=7$. Then $n=35$, $z=24$.

$e=5$ (so e , z relatively prime).

$d=29$ (so $ed-1$ exactly divisible by z).

encrypting 8-bit messages.



Why does RSA work?

- must show that $c^d \bmod n = m$, where $c = m^e \bmod n$
- fact: for any x and y : $x^y \bmod n = x^{(y \bmod z)} \bmod n$
 - where $n = pq$ and $z = (p-1)(q-1)$
- thus,
$$\begin{aligned}c^d \bmod n &= (m^e \bmod n)^d \bmod n \\&= m^{ed} \bmod n \\&= m^{(ed \bmod z)} \bmod n \\&= m^1 \bmod n \\&= m\end{aligned}$$

RSA: another important property

The following property will be *very* useful later:

$$\underbrace{K_B^-(K_B^+(m))}_{\text{use public key first, followed by private key}} = m = \underbrace{K_B^+(K_B^-(m))}_{\text{use private key first, followed by public key}}$$

use public key
first, followed
by private key

use private key
first, followed
by public key

result is the same!

Why $K_B^-(K_B^+(m)) = m = K_B^+(K_B^-(m))$?

follows directly from modular arithmetic:

$$\begin{aligned}(m^e \bmod n)^d \bmod n &= m^{ed} \bmod n \\ &= m^{de} \bmod n \\ &= (m^d \bmod n)^e \bmod n\end{aligned}$$

Why is RSA secure?

- suppose you know Bob's public key (n,e) . How hard is it to determine d ?
- essentially need to find factors of n without knowing the two factors p and q
 - fact: factoring a big number is hard

RSA in practice: session keys

- exponentiation in RSA is computationally intensive
- DES is at least 100 times faster than RSA
- use public key crypto to establish secure connection, then establish second key – symmetric session key – for encrypting data

session key, K_s

- Bob and Alice use RSA to exchange a symmetric session key K_s
- once both have K_s , they use symmetric key cryptography

What could go wrong?

- What was wrong with the toy example?
 - Eve can easily find the encryption of each letter and use that as a key to Alice's message
 - Even without knowing the public key, can use statistics to find likely messages
 - Like cryptogram puzzles

How it should really happen

- p and q should be at least 512 bits each
 - N at least 1024 bits
- The message “HELLO WORLD” would be converted into one very large integer
- That integer would be raised to the public/private exponent
- For short message, pad them with a random string

Public Key Paradigm

- Genkey(some info)
 - Creates K_{pub} and K_{priv}
- Encrypt with K_{pub}
- Decrypt with K_{priv}
- Certificate binds key to individual

Outline

- Our Actors
- Public Key Cryptography
 - Diffie Hellman Key Exchange
 - RSA Algorithm
- Authentication
- Deployed in wireless and mobile networks

Using Public-Key Cryptography



We'll (next) see *lots* of non-confidentiality uses of public-key cryptography!

- authentication
- digital signatures
- message integrity
- symmetric-key agreement

Authentication

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap1.0: Alice says “I am Alice”



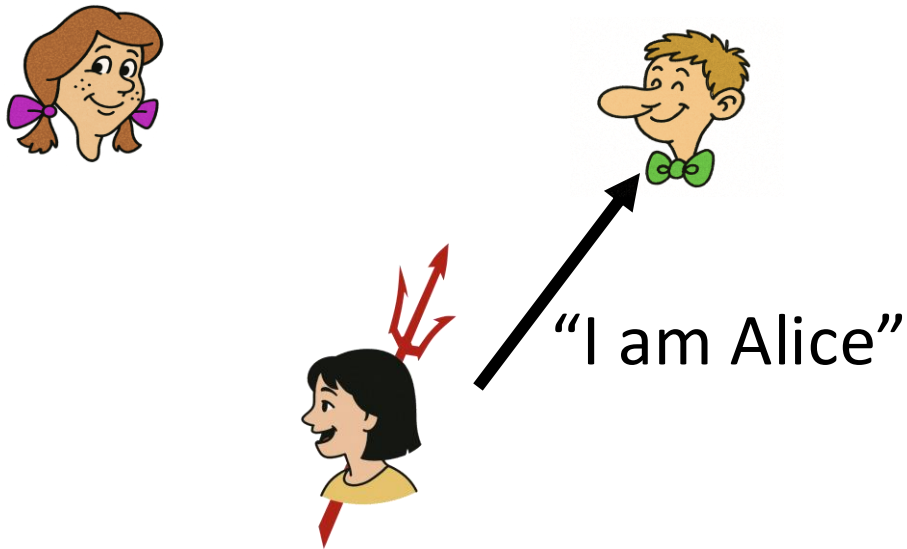
failure scenario??



Authentication

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap1.0: Alice says “I am Alice”



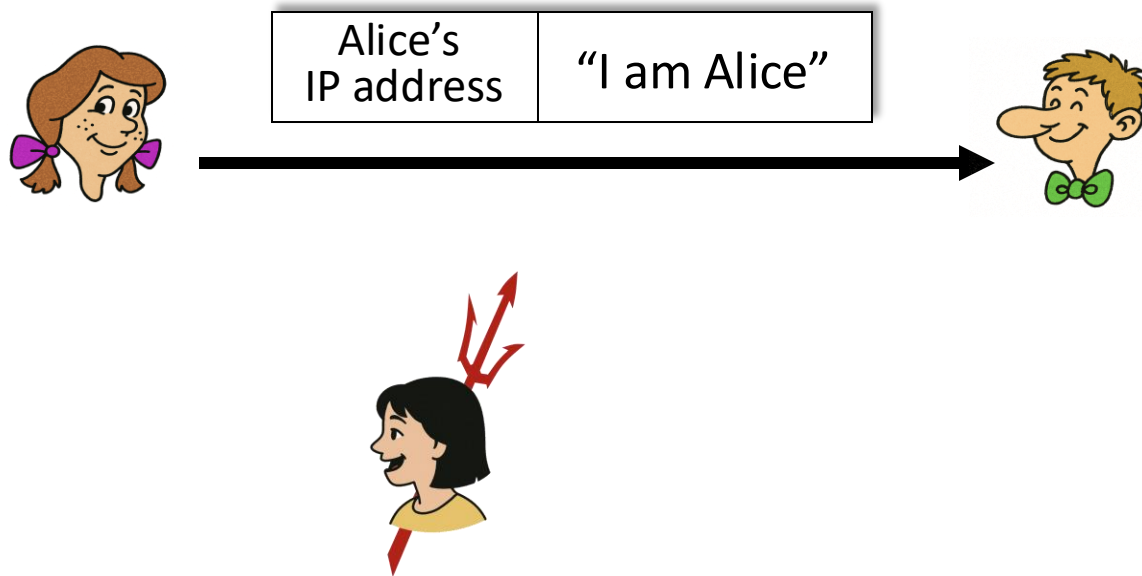
*in a network, Bob
can not “see”
Alice, so Trudy
simply declares
herself to be Alice*



Authentication: another try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap2.0: Alice says “I am Alice” in an IP packet containing her source IP address

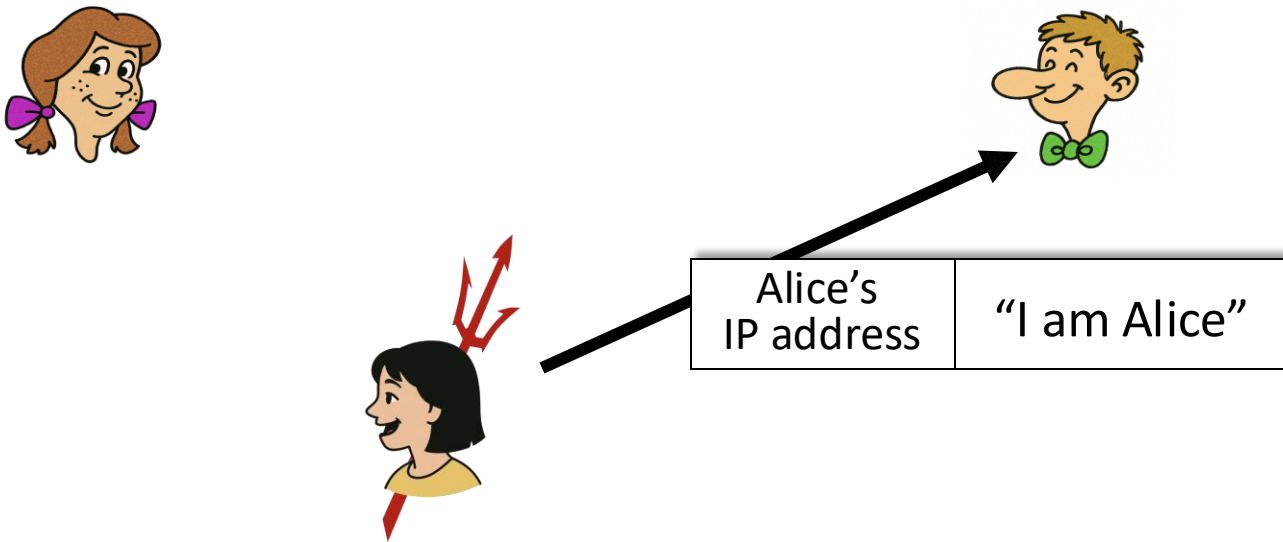


failure scenario??

Authentication: another try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap2.0: Alice says “I am Alice” in an IP packet containing her source IP address

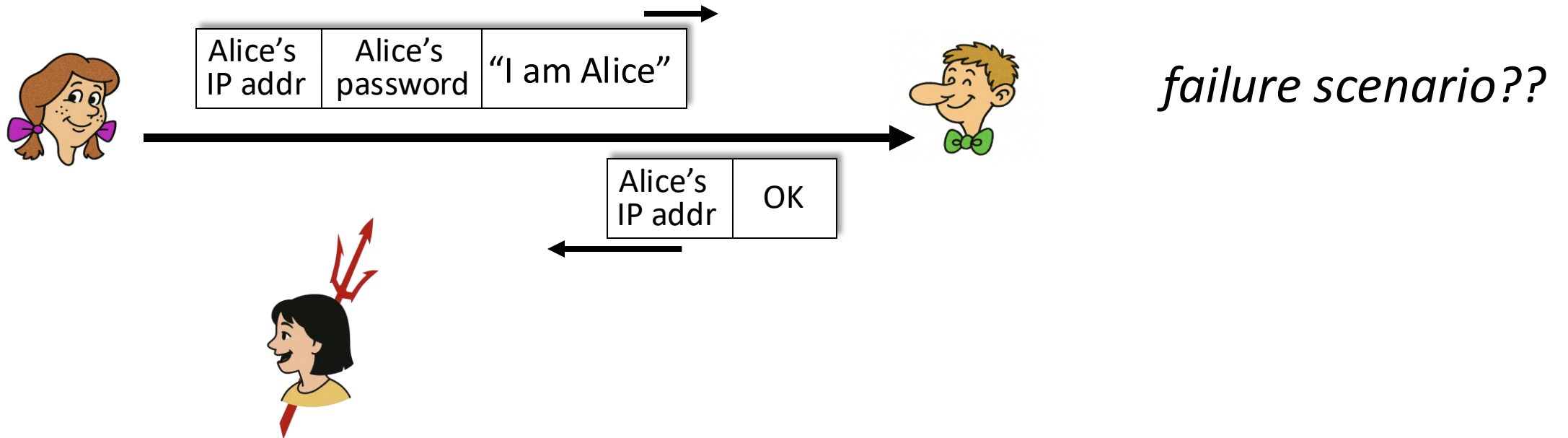


*Trudy can create
a packet “spoofing”
Alice’s address*

Authentication: a third try

Goal: Bob wants Alice to “prove” her identity to him

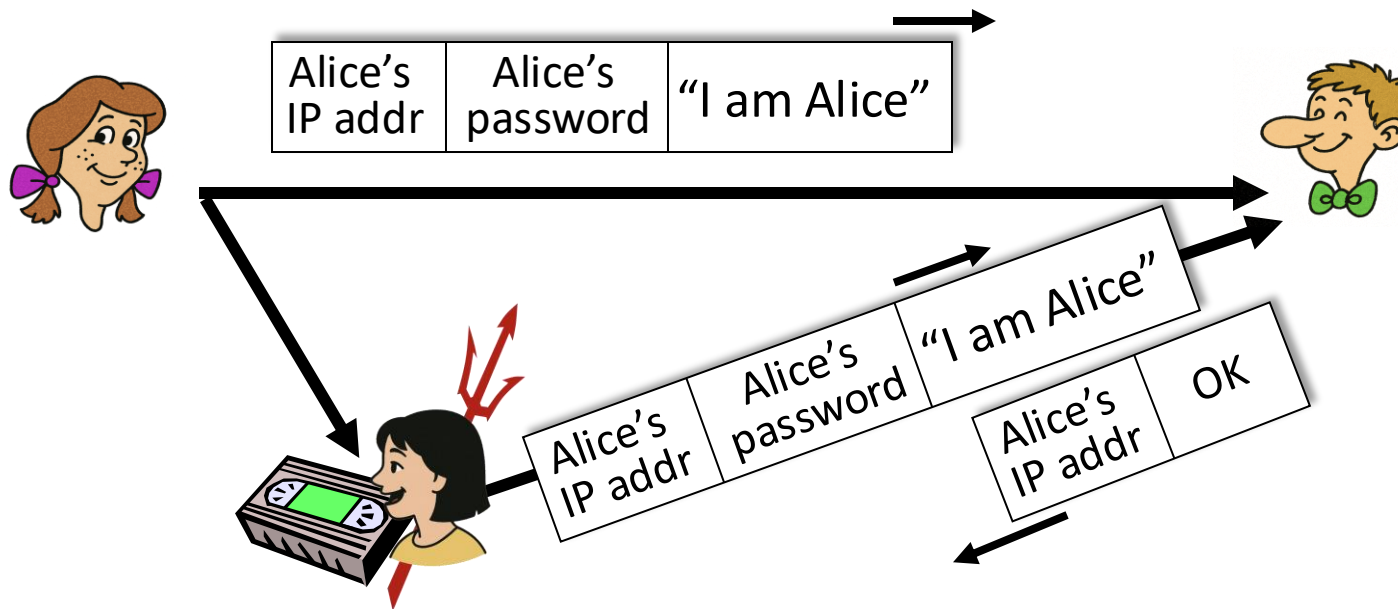
Protocol ap3.0: Alice says “I am Alice” and sends her secret password to “prove” it.



Authentication: a third try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap3.0: Alice says “I am Alice” and sends her secret password to “prove” it.

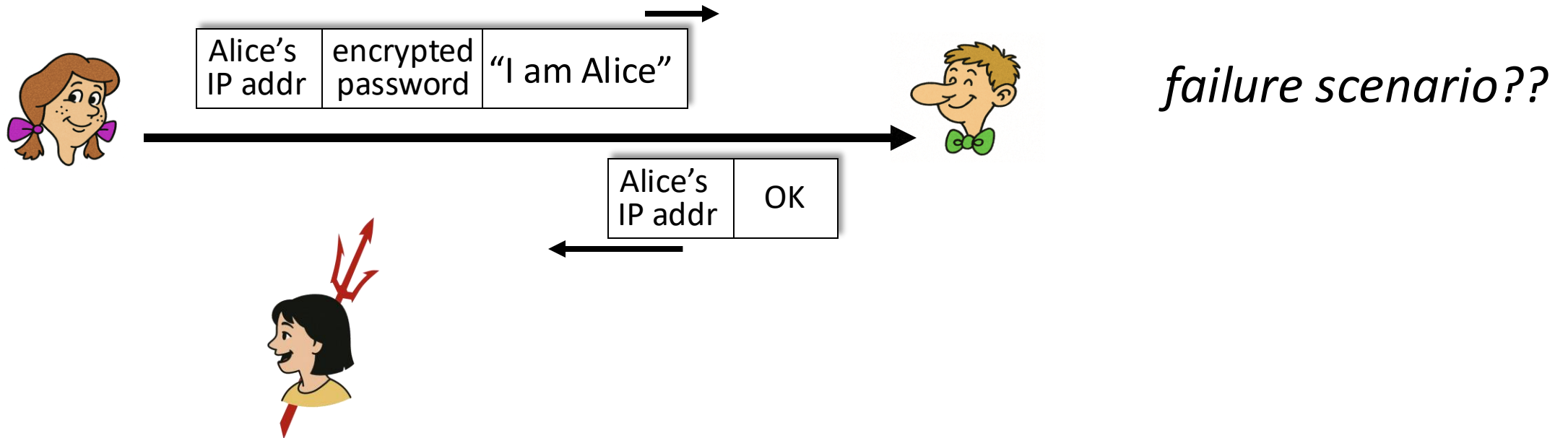


*playback attack:
Trudy records
Alice's packet
and later
plays it back to Bob*

Authentication: a modified third try

Goal: Bob wants Alice to “prove” her identity to him

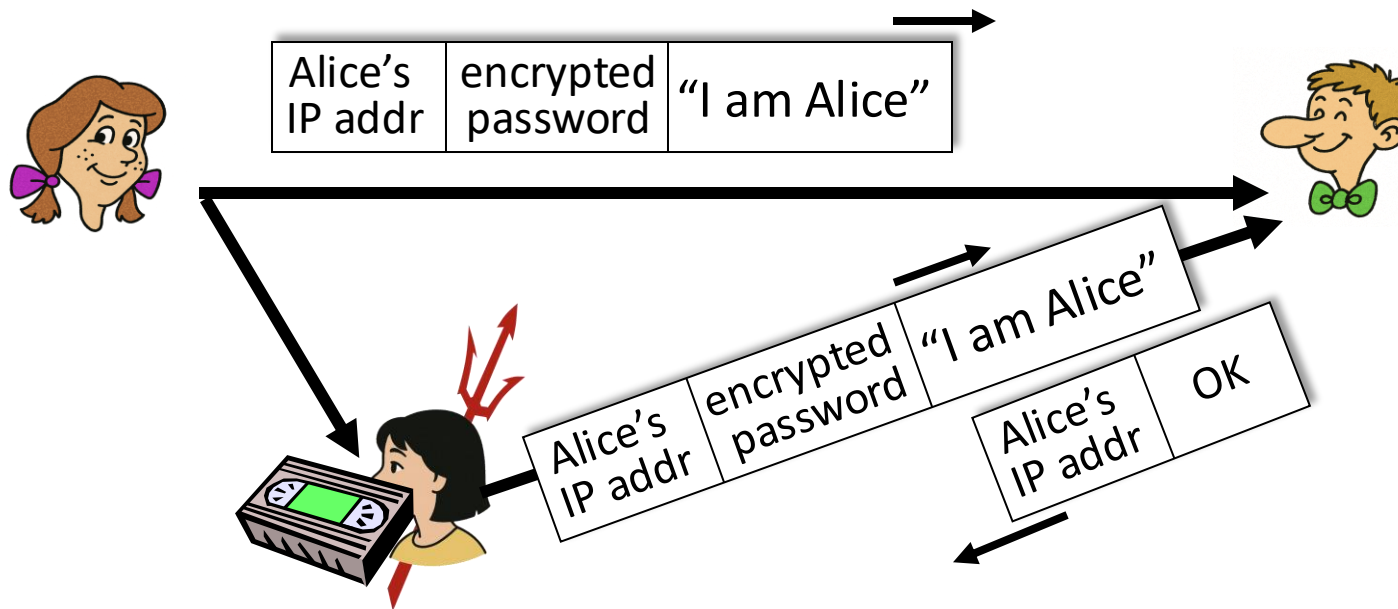
Protocol ap3.0: Alice says “I am Alice” and sends her encrypted secret password to “prove” it.



Authentication: a modified third try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap3.0: Alice says “I am Alice” and sends her encrypted secret password to “prove” it.



playback attack still works: Trudy records Alice's packet and later plays it back to Bob

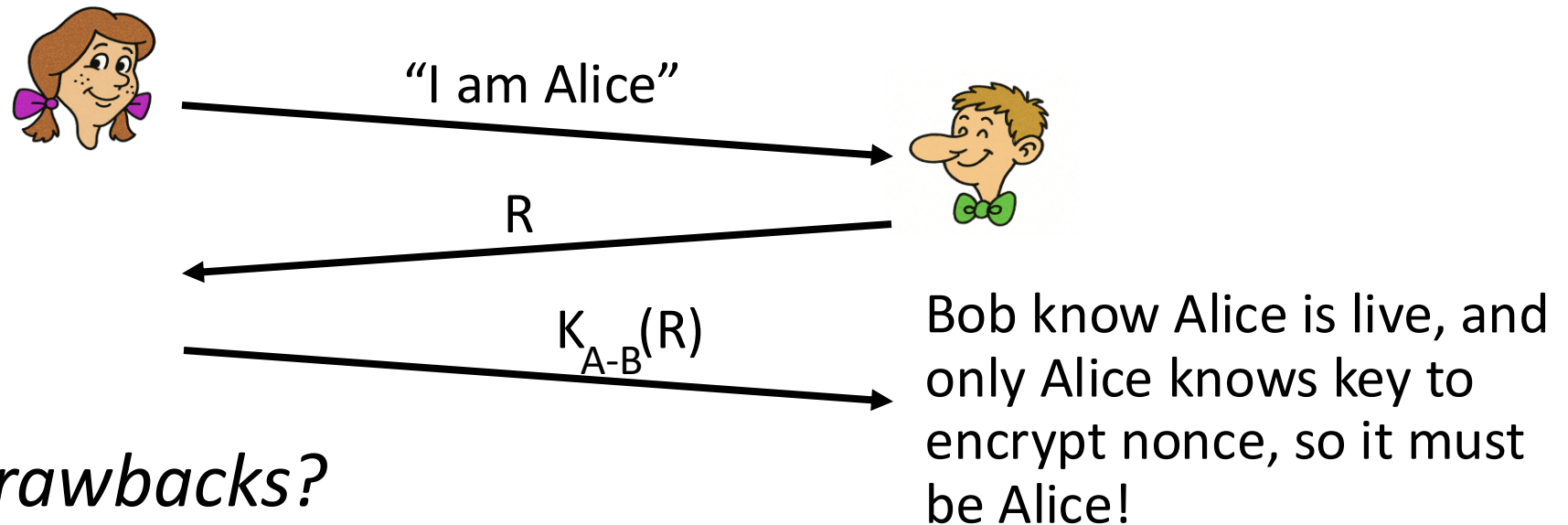
Authentication: a fourth try

Goal: avoid playback attack

nonce: number (R) used only **once-in-a-lifetime**

protocol ap4.0: to prove Alice “live”, Bob sends Alice nonce, R

- Alice must return R, encrypted with shared secret key

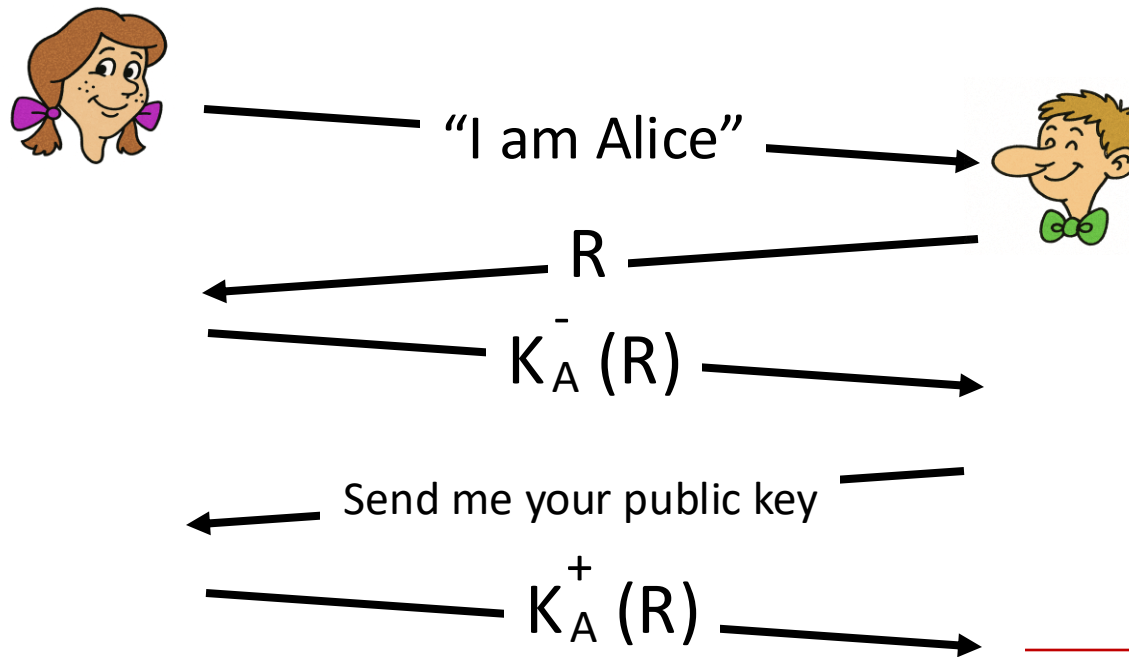


Failures, drawbacks?

Authentication: ap5.0

ap4.0 requires shared symmetric key - can we authenticate using public key techniques?

ap5.0: use nonce, public key cryptography



Bob computes

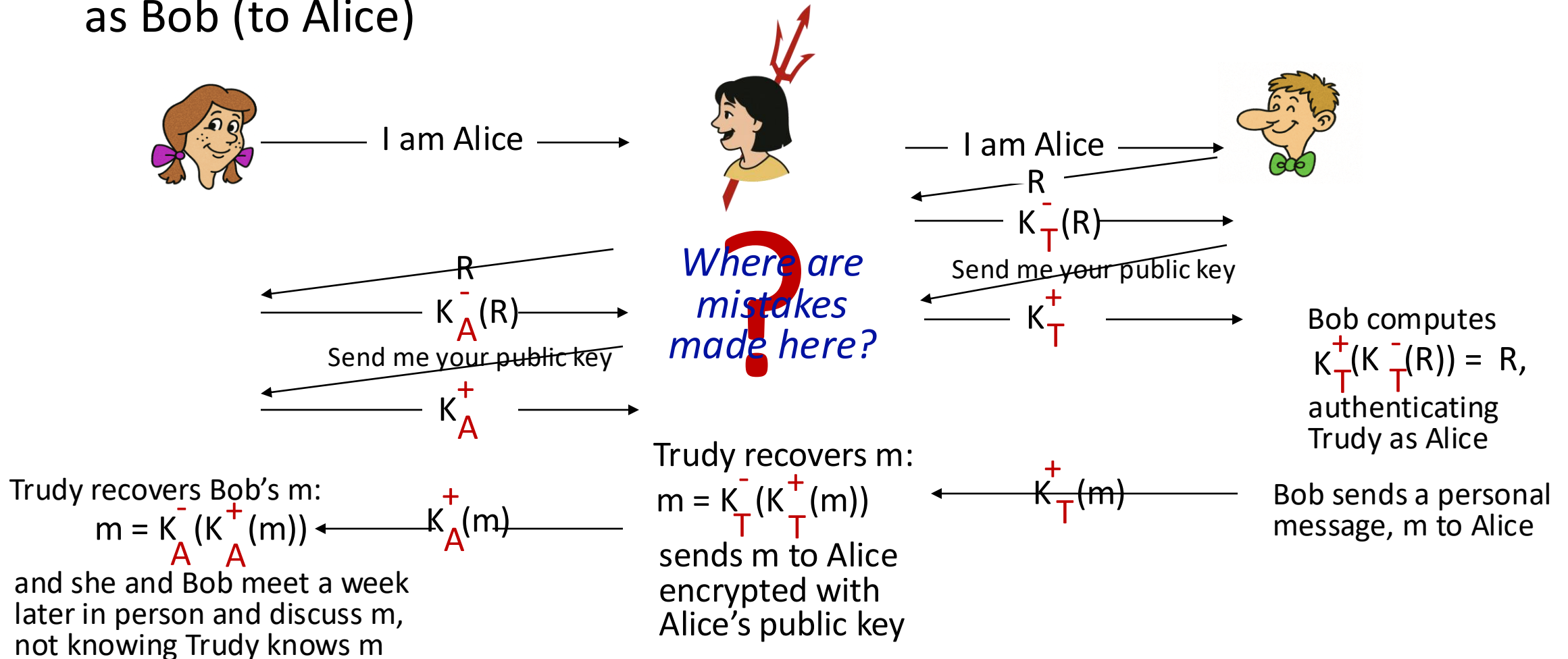
$$K_A^+ (K_A^-(R)) = R$$

and knows only Alice could have the private key, that encrypted R such that

$$K_A^+ (K_A^-(R)) = R$$

Authentication: ap5.0 – there's still a flaw!

man (or woman) in the middle attack: Trudy poses as Alice (to Bob) and as Bob (to Alice)



Outline

- Elliptic Curve Cryptography
- Deployment in wireless and mobile networks

Why Elliptic Curve Cryptography?

- Shorter Key Length
- Lesser Computational Complexity
- Low Power Requirement
- More Secure

Comparable Key Sizes for Equivalent Security

Symmetric Encryption (Key Size in bits)	RSA and Diffie-Hellman (modulus size in bits)	ECC Key Size in bits
56	512	112
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	512

More Advanced

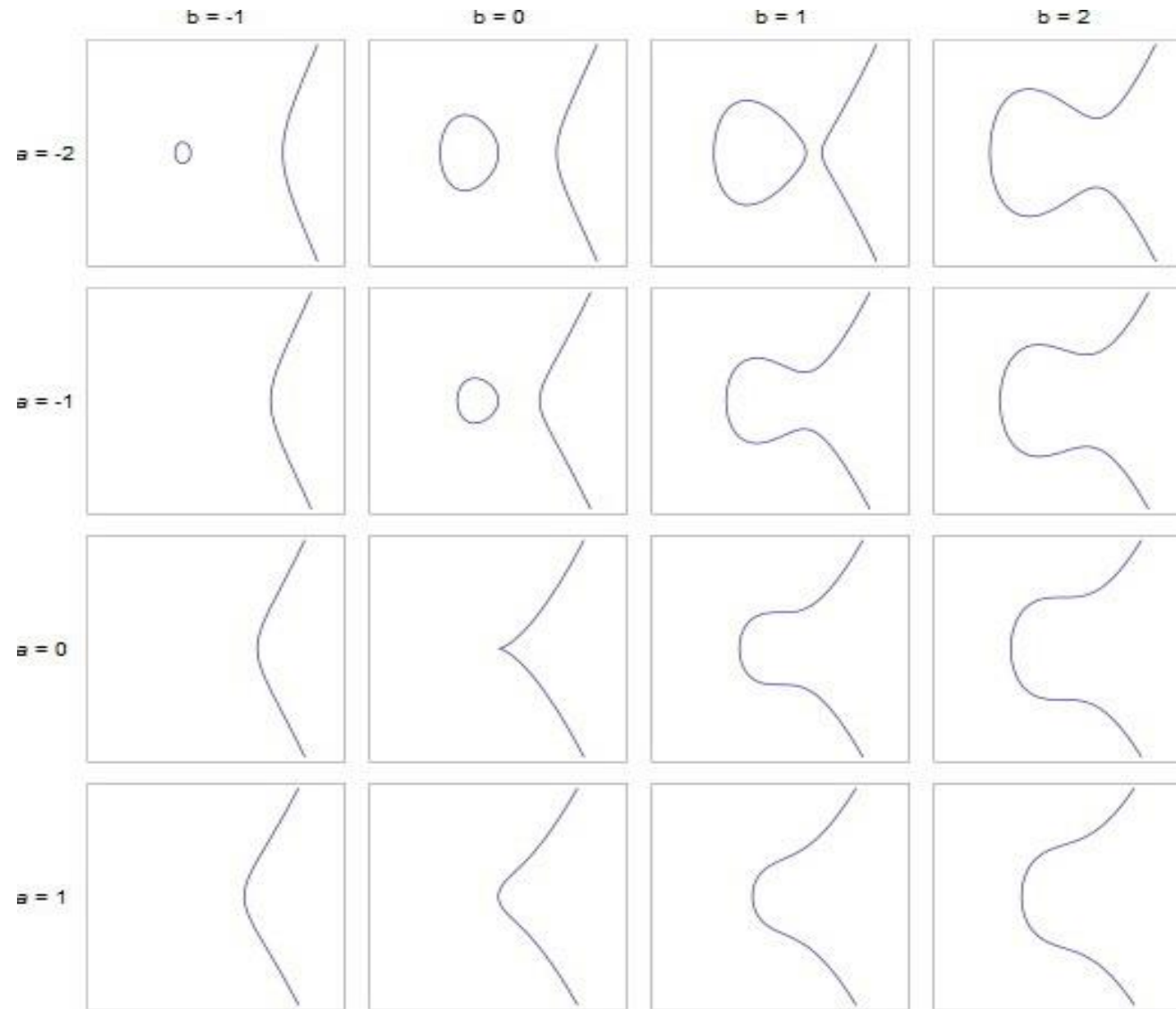
■ What are Elliptic Curves?

- Curve with standard form

$$y^2 = x^3 + ax + b \quad a, b \in \mathbb{R}$$

■ Characteristics of Elliptic Curve

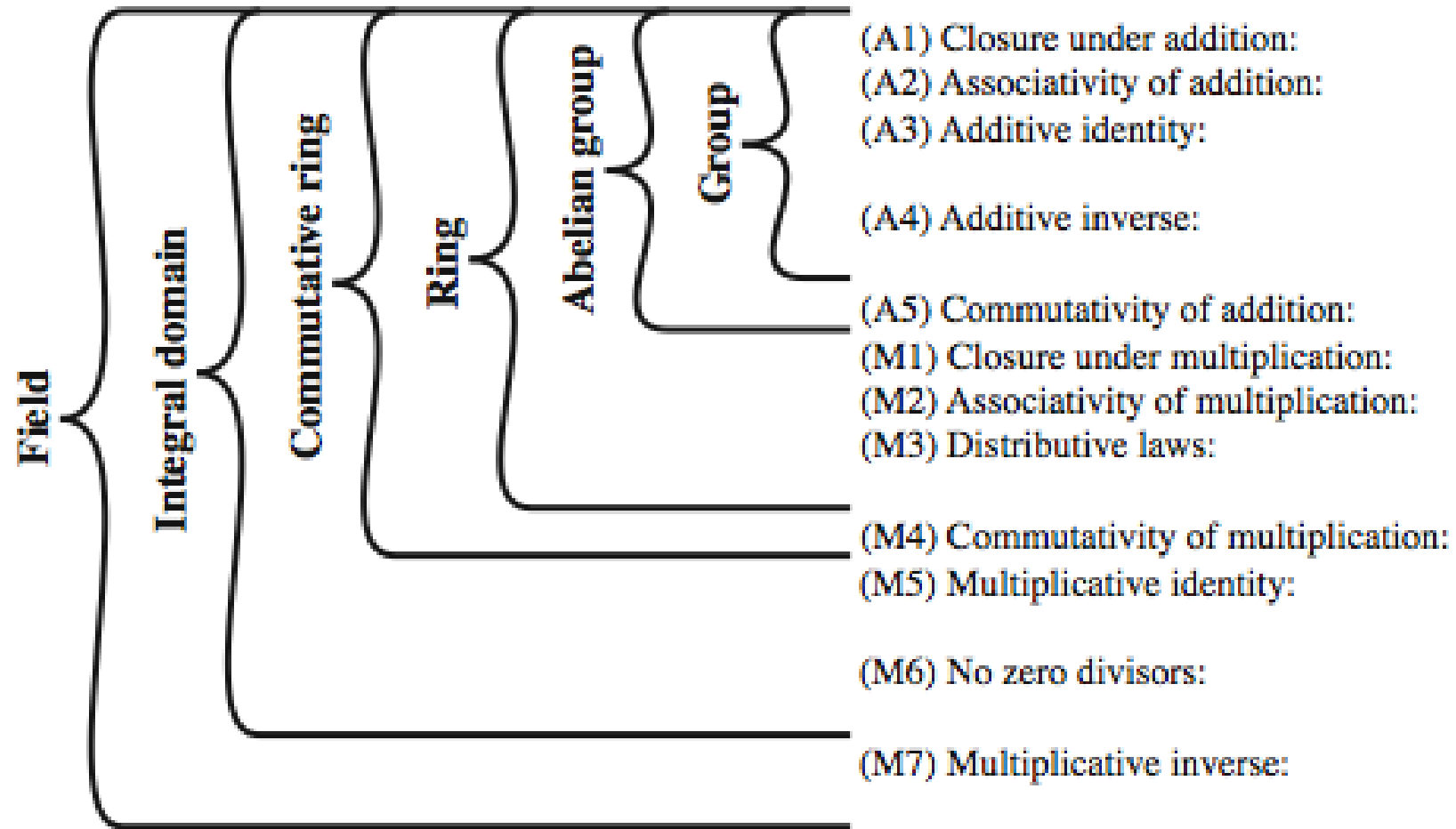
- Forms an abelian group
- Symmetric about the x-axis
- *Point at Infinity* acting as the identity element



Examples of Elliptic Curves

Finite Fields

- aka Galois Field
- $GF(p^n)$ = a set of integers $\{0, 1, 2, \dots, p^n - 1\}$
where p is a prime, n is a positive integer
- It is denoted by $\{F, +, \times\}$
where $+$ and $\times$ are the group operators



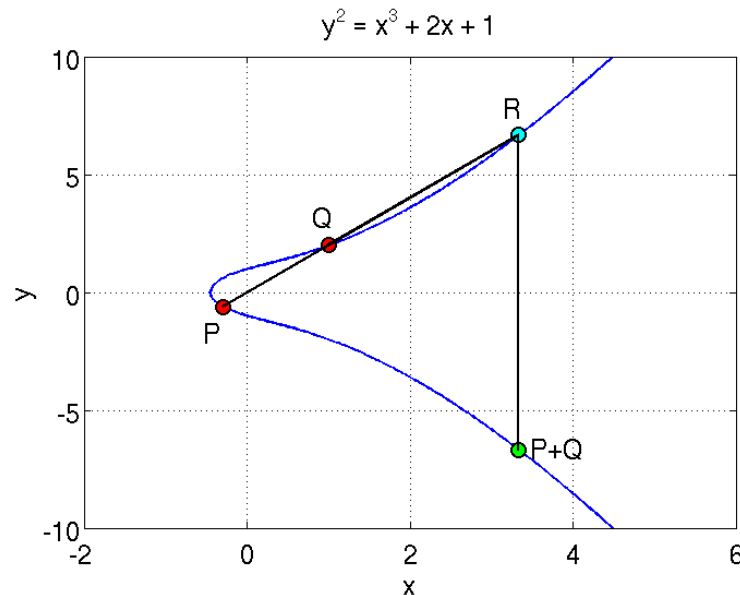
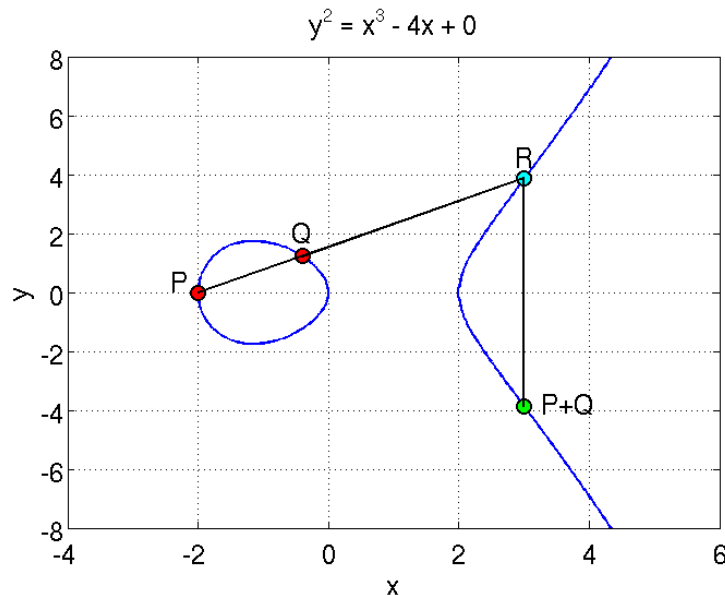
Group, Ring, Field

What is Elliptic Curve Cryptography?

■ Implementing Group Operations

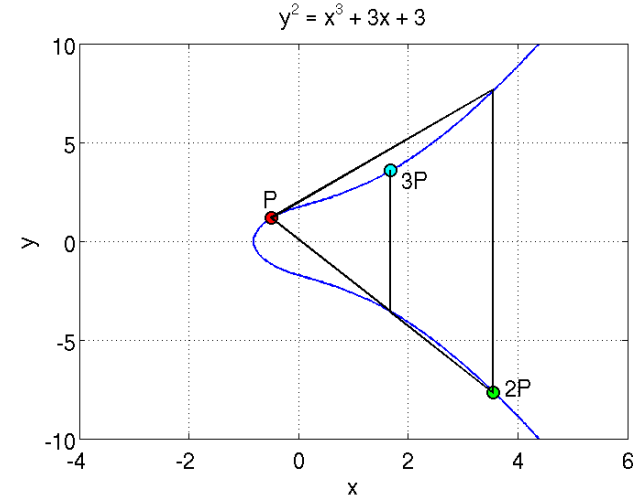
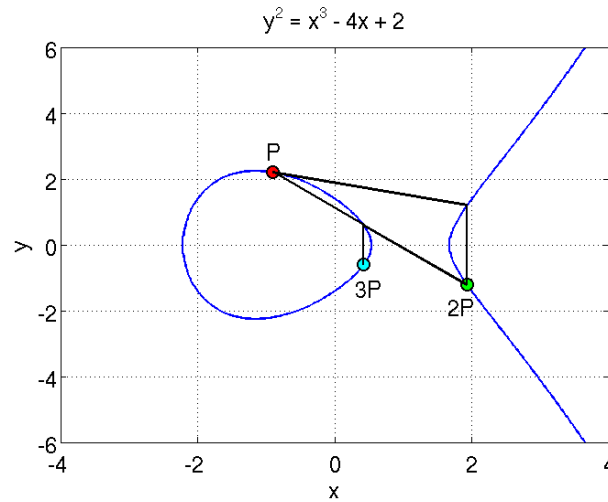
- Main operations - point addition and point multiplication
- Adding two points that lie on an Elliptic Curve – results in a third point on the curve
- Point multiplication is repeated addition
- If P is a known point on the curve (aka Base point; part of domain parameters) and it is multiplied by a scalar k , $Q=kP$ is the operation of adding $P + P + P + P \dots + P$ (k times)
- Q is the resulting public key and k is the private key in the public-private key pair

What is Elliptic Curve Cryptography?



- Adding two points on the curve
- P and Q are added to obtain P+Q which is a reflection of R along the X axis

What is Elliptic Curve Cryptography?



- A tangent at P is extended to cut the curve at a point; its reflection is 2P
- Adding P and 2P gives 3P
- Similarly, such operations can be performed as many times as desired to obtain $Q = kP$

What is Elliptic Curve Cryptography?

■ Discrete Log Problem

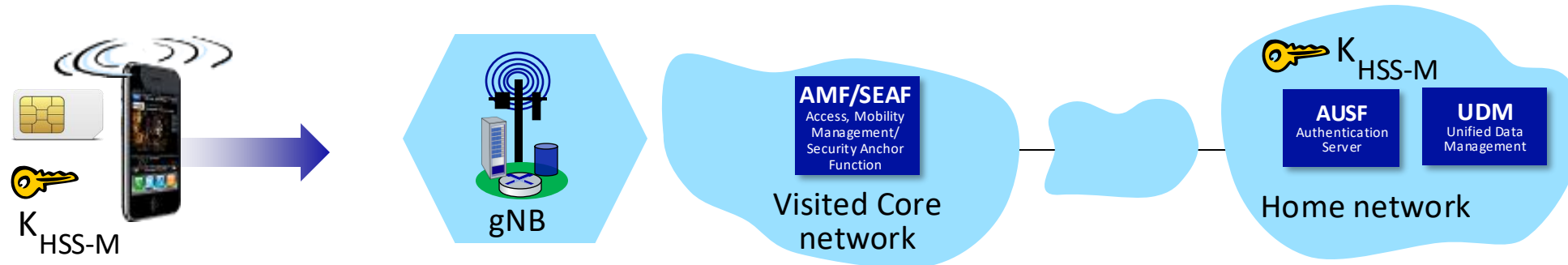
- The security of ECC is due the intractability or difficulty of solving the inverse operation of finding k given Q and P
- This is termed as the discrete log problem
- Methods to solve include brute force and Pollard's Rho attack both of which are computationally expensive or unfeasible
- The version applicable in ECC is called the Elliptic Curve Discrete Log Problem
- Exponential running time

Chapter 8 outline

- What is network security?
- Principles of cryptography
- Authentication, digital signatures, message integrity, shared key agreement
- Securing e-mail
- Securing TCP connections: TLS
- Network layer security: IPsec
- **Security in wireless and mobile networks**
 - 802.11 (WiFi)
 - 4G/5G
- Operational security: firewalls and IDS

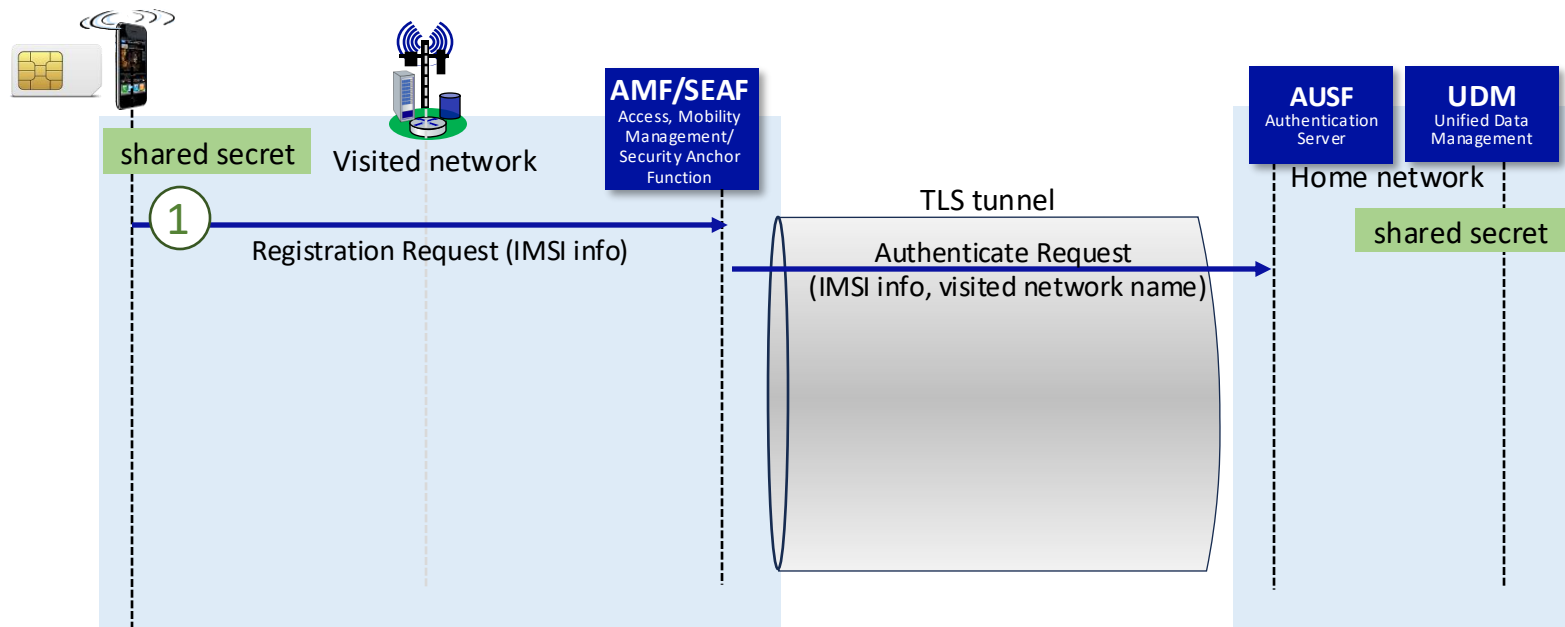


Authentication, encryption in 5G



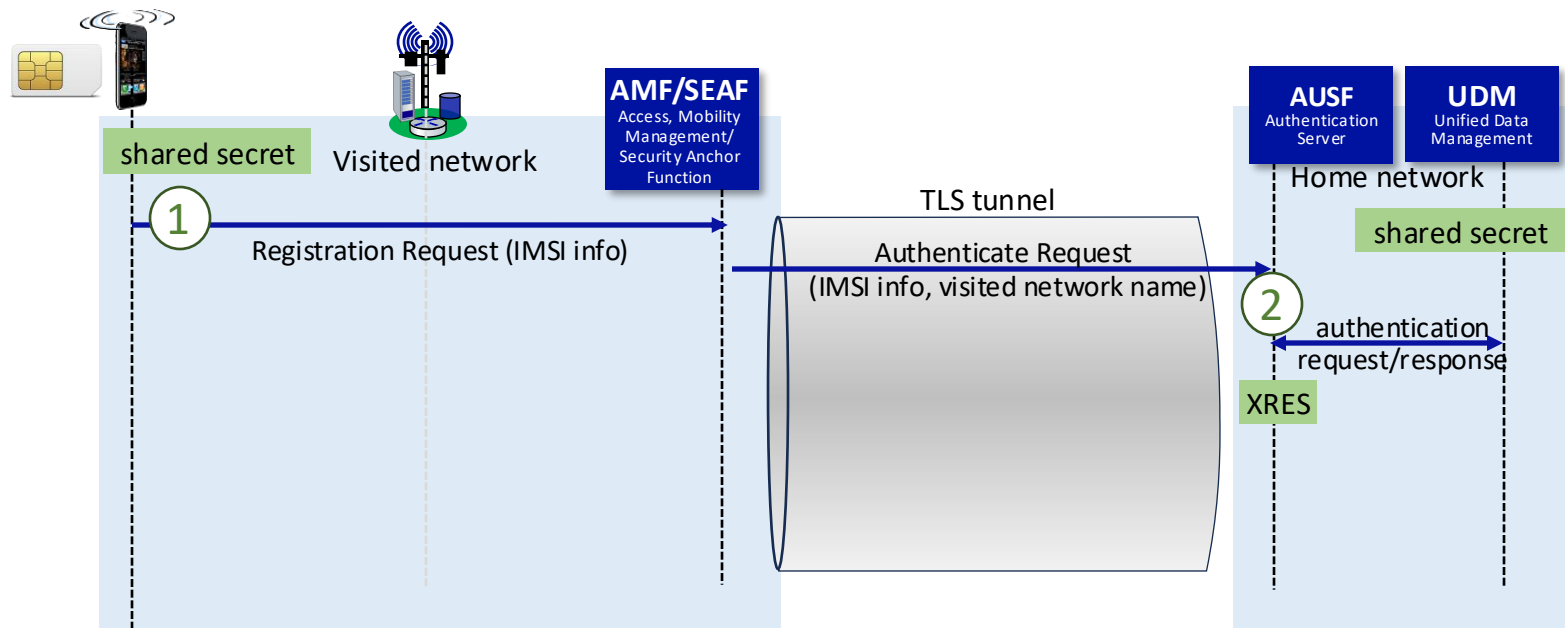
- arriving mobile must:
 - associate with BS: (establish) communication over 4G wireless link
 - authenticate itself to network, and authenticate network
- notable differences from WiFi
 - UE's SIM provides global identity, contains shared keys K_{HSS-M}
 - services in visited network depend on (paid) service subscription in home network

Authentication, encryption in 5G



- ①
- UE sends Registration Request message to visited AMF with identity (IMSI) info
 - visited AMF locates AUSF in device's *home* network, establishes secure TLS tunnel to home AUSF (all messages authenticated, encrypted)

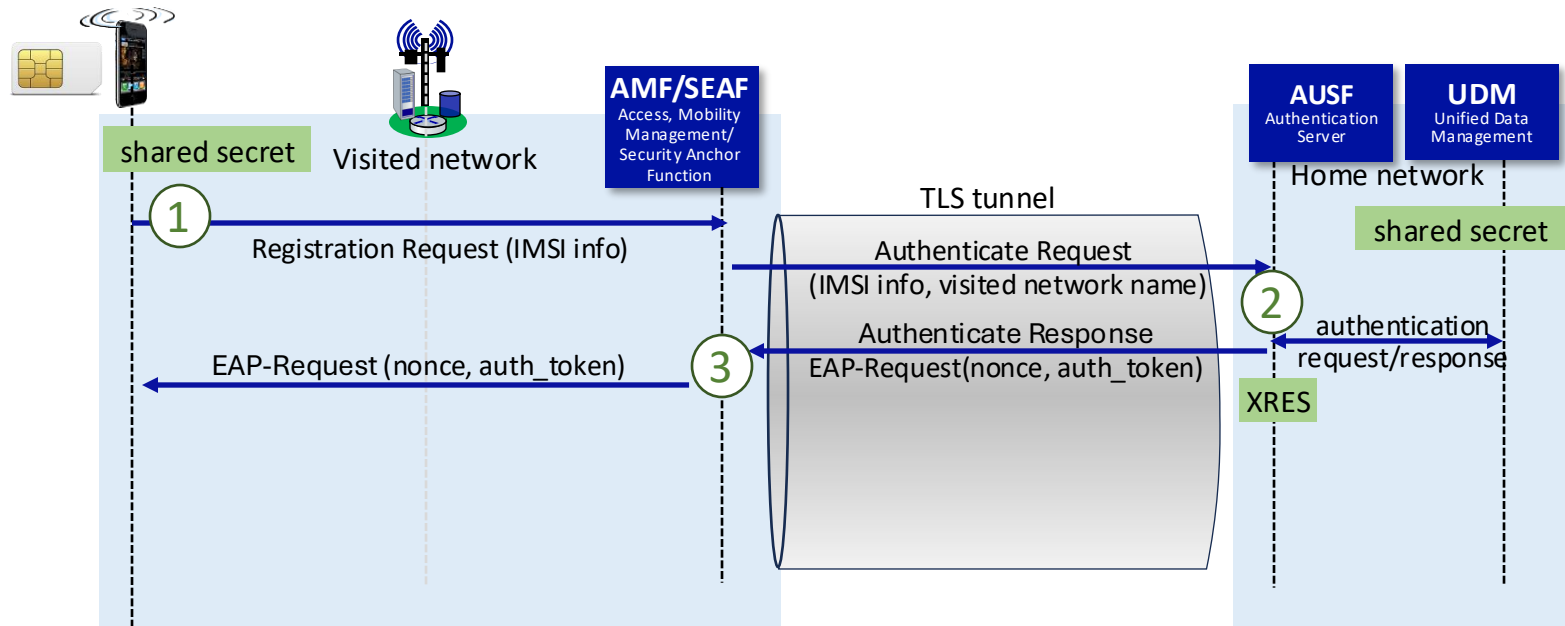
Authentication, encryption in 5G



②

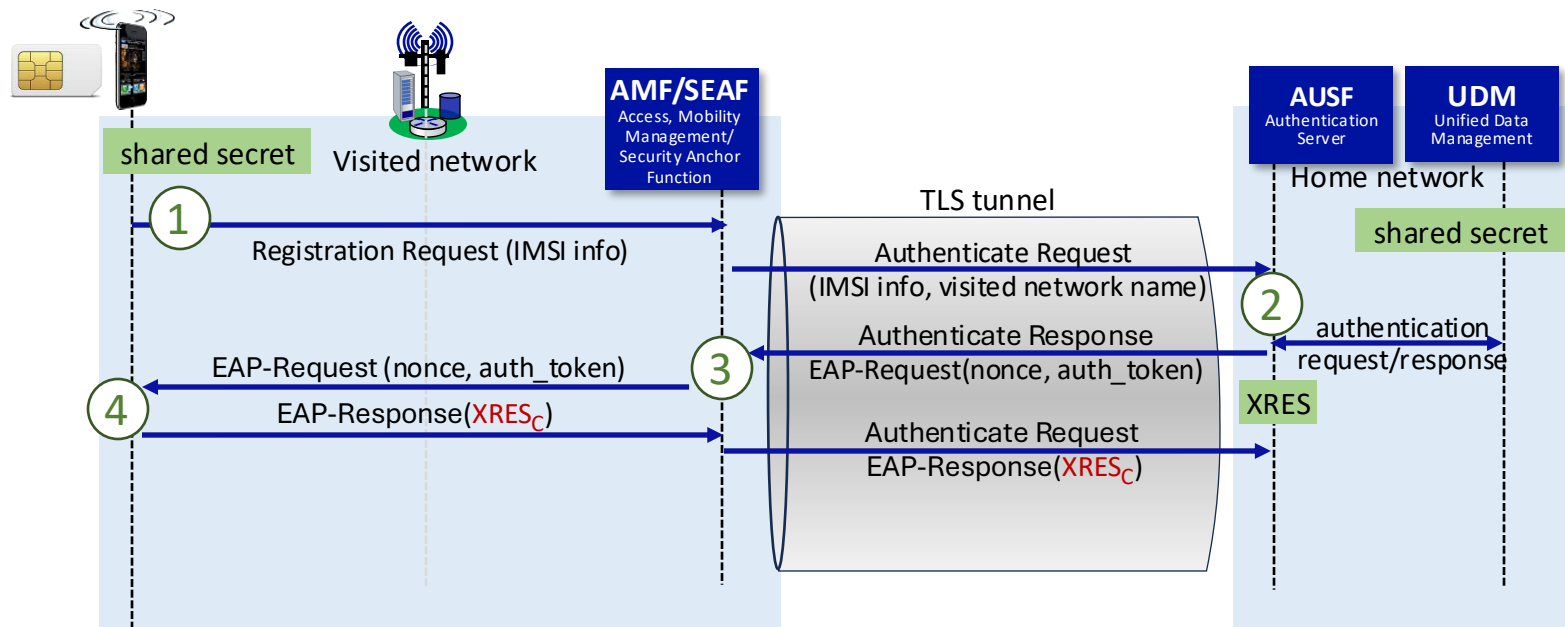
- home AUSF contacts home UDM
- home UDM creates authentication token (using UE's key) and nonce, and signs (will prove to UE that home UDM generated token)
- UDM computes, saves expected response (XRES) that UE will compute

Authentication, encryption in 5G



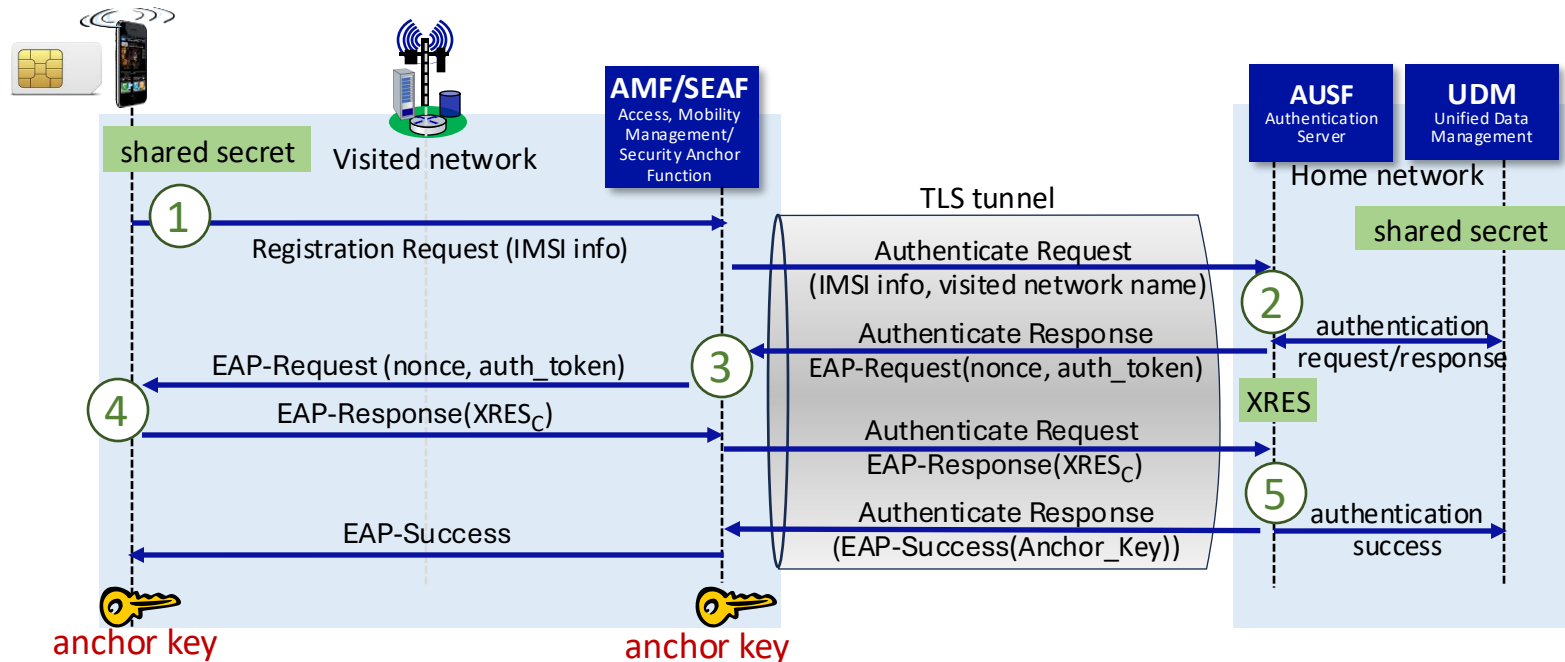
- ③ ■ home AUSF sends authentication token, nonce to AMF, which forwards to UE

Authentication, encryption in 5G



- UE receives authentication token, nonce
- Given shared secret on its SIM, determines token is indeed from its home AUSF
- UE computes its expected response ($XRES_C$) sends to home AUSF

Authentication, encryption in 5G

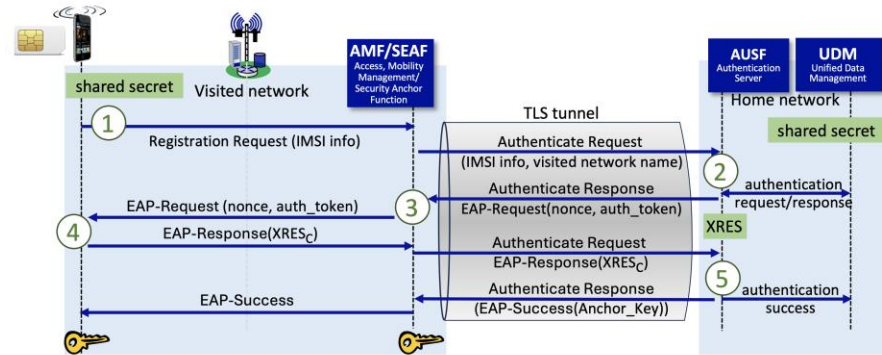


Moment of truth:

5

- AUSF computes $XRES_C$ and compares with XRES it computed earlier
- **authentication success:** home UDM updated
 - home AUSF generates **anchor key** and sends it to visited AMF (device will compute anchor key on its own)
 - AMF, device use anchor key to secure message exchange in visited network

5G Authentication: observations



- *home* AUSF makes authentication comparison (used by MME in visited network in 4G)
- *home* AUSF generates anchor key to be used in visited network
 - informs UE and AMF and gNB in visited network
- note *use of crypto keys on UE's SIM* (known only to UE and home AUSF)